

2025—2026 学年第二学期

桥梁工程课程设计报告

基于 Manim 的桥梁工程教学动画生成 与交互审阅系统

课程名称： 桥梁工程课程设计

项目组成： 项目 1：教学动画生成器；项目 2：交互审阅子系统

学生姓名： _____

学 号： _____

专业班级： _____

指导教师： _____

完成日期： 2026 年 6 月

课程设计成果文档

摘 要

本课程设计面向桥梁工程知识表达与设计成果展示需求，开发了一套由“教学动画生成端”和“交互式审阅端”组成的本地化软件平台。系统以桥梁结构、构件关系、荷载传递和施工过程等内容为表达对象，将传统课程设计中依赖静态图纸与文字说明的成果，扩展为可分镜、可渲染、可审阅、可回滚的动态教学视频。项目 1 采用 Electron、FastAPI、Python 与 Manim Community Edition 构建，完成从问题定框、教学大纲、分镜设计、Manim 代码生成、静态检查、渲染修复到字幕、音频和项目导出的全流程；项目 2 以 React、FastAPI 和结构化文件为基础，围绕视频时间轴提供编号批注、分镜匹配、上下文包与 Agent Task 生成、局部重渲染、版本记录和回滚功能。

设计中重点解决了复杂工程概念如何转化为可执行视觉方案、长流程如何保持状态可追溯、生成代码如何在渲染前进行约束、用户反馈如何精确定位到分镜及对象等问题。系统使用有向无环节点工作流表达数据依赖，以问题定框和流水线事件记录维持任务一致性，以静态检查和最多三轮修复降低无效渲染，以 SHOT_ID 与 OBJECT_ID 保证局部修改边界。斜拉桥示例将桥面、桥塔和斜拉索的构造关系及“车辆荷载—桥面—斜拉索—桥塔—地基”的传力路径组织为三段动画，验证了系统服务桥梁工程课程设计表达的可行性。

测试结果表明，项目 1 后端 46 项自动化测试全部通过；项目 2 完成静态自检、前后端构建、视频接口、批注保存、分镜匹配、上下文生成、任务生成、渲染、修订与回滚验证。该平台并不替代桥梁结构计算，而是将课程设计中的工程理解、表达组织和成果复核转化为可重复执行的软件工作流，为桥梁工程教学可视化及人机协同审阅提供了具有扩展性的实现基础。

关键词：桥梁工程；Manim；教学动画；节点工作流；分镜管理；交互批注；版本回滚

目 录

第一章 项目背景与研究意义 1

第二章 需求分析 4

第三章 系统总体设计 8

第四章 系统功能设计 14

第五章 核心模块实现 21

第六章 关键技术分析 29

第七章 系统运行与测试 35

第八章 项目特色与创新点 40

第九章 存在的问题与改进方向 43

第十章 总结与展望 46

参考文献 48

致谢 49

第一章 项目背景与研究意义

1.1 课程设计背景

桥梁工程课程设计要求学生综合运用结构体系、构件受力、荷载传递、施工方法与工程表达等知识，将课堂中的分散概念组织为可解释、可校核的设计成果。常规报告以计算书、结构示意图和施工图为主，能够准确记录结果，但对斜拉索逐步受力、荷载沿构件传递、结构体系对比等具有时间顺序和空间关系的知识，静态媒介往往需要大量文字才能说明。

本项目选择“桥梁工程知识的动态可视化与交互审阅”作为课程设计的软件实现方向。其工程目标不是代替规范验算、有限元分析或施工图设计，而是建立一条把工程理解转化为教学动画的生产线：先明确桥梁主题和教学目标，再形成分镜和视觉计划，随后生成并渲染 Manim 代码，最后通过视频批注对局部画面进行迭代。这一定位与模板中强调的“核心功能、算法流程、可视化结果和交互记录”相吻合。

1.2 桥梁工程表达中的实际问题

表 1-1 桥梁工程可视化表达问题与设计对策

问题	传统方式局限	本系统对应策略
结构体系复杂	构件多、关系密集，单图难以同时表达构造与受力	按分镜逐层出现桥面、桥塔、拉索及荷载箭头
传力路径抽象	文字描述缺少方向和时间顺序	用颜色、箭头和动画顺序显示荷载传递
成果修改成本高	一处改动常需重做整段演示	按 SHOT_ID 定位并仅重渲染目标分镜
反馈表达含糊	“这里不清楚”难以对应代码对象	视频画布编号批注并映射 OBJECT_ID
过程难追溯	最终视频无法说明生成与修改依据	保存流水线事件、上下文包、修订和版本快照

1.3 研究意义

从教学层面看，系统把构件识别、受力分析和体系对比转化为连续视觉叙事，有利于学生用“结构—作用—响应”的逻辑解释桥梁工程问题。从工程表达层面看，分镜与对象标识促使

设计者在编码前明确每一画面的目的，避免仅追求动画效果而忽视工程含义。从软件工程层面看，生成端和审阅端分离，使批量生产与精细修改各自保持清晰边界。

此外，本项目体现了课程设计过程性评价的要求。问题定框、分镜文件、代码、渲染日志、批注、任务文本和修订记录共同构成可审计证据链，教师能够追踪“为什么这样设计、如何实现、出现何种失败、怎样改进”，而不只是观看最终视频。

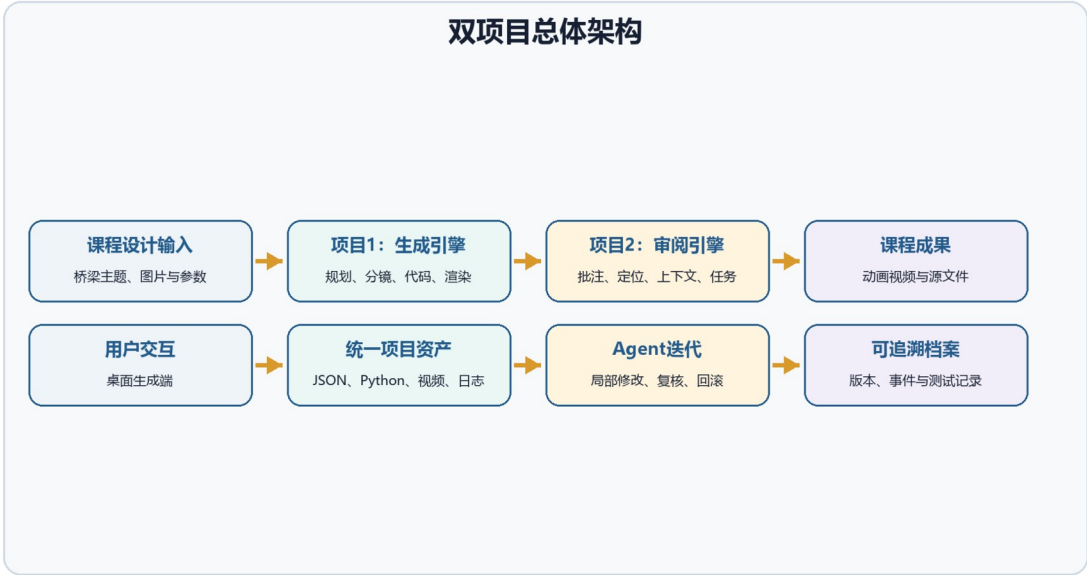


图 1-1 双项目组成及其在课程设计中的关系

第二章 需求分析

2.1 用户与使用场景

系统的主要使用者包括完成课程设计的学生、进行过程指导的教师以及承担局部代码修改的智能代理。学生负责输入桥梁主题、检查分镜、观看视频并提出批注；教师关注工程概念是否正确、表达是否清楚以及修改过程是否可追溯；Agent 依据结构化任务在严格边界内修改代码。三类角色通过项目文件而非黑箱会话协作。

2.2 功能需求



图 2-1 系统功能模块划分

表 2-1 主要功能需求

编号	需求	验收要点
F01	桥梁主题与素材输入	支持提示词、图片、时长、质量和输出目录
F02	教学规划与分镜生成	输出目标明确、字段完整的结构化分镜
F03	Manim 代码与渲染	每个分镜生成可检查、可独立渲染的 Scene
F04	字幕、音频与视频合成	维护片段时长、旁白、字幕和合成结果
F05	节点工作流	节点端口类型校验、拓扑执行、状态保存
F06	自定义节点	从 UTF-8 JSON 注册且不得覆盖内置节点
F07	视频批注	矩形、箭头、画笔和编号标签可组合提交

F08	局部分镜修改	按时间匹配 shot，仅修改对应 Scene
F09	版本与回滚	成功修订有记录，失败可恢复上一版本

2.3 非功能需求

可靠性方面，渲染前必须先完成 Python 语法检查，渲染进程必须设置超时，后台重启后不得把中断任务伪装为仍在执行。可维护性方面，AI 路由、工作流、渲染、媒体和项目管理应分层组织。可移植性方面，核心数据采用 JSON、Markdown、Python 和 MP4 等通用格式，并提供本地便携运行方式。安全性方面，工作流 JSON 不保存 API 密钥，审阅子系统不直接调用大模型。

2.4 数据与边界约束

系统输入不是任意自然语言到任意视频的无约束映射。问题定框文件固定主题、时长、质量、图片优先级和节奏要求；分镜 Schema 固定场景编号、旁白、视觉计划和对象；工作流连接必须满足端口类型；审阅修改必须保留 SHOT_ID 和 OBJECT_ID，并禁止混入 ManimGL 语法。上述约束使生成结果从“可能可用”转向“可检查、可定位、可回退”。

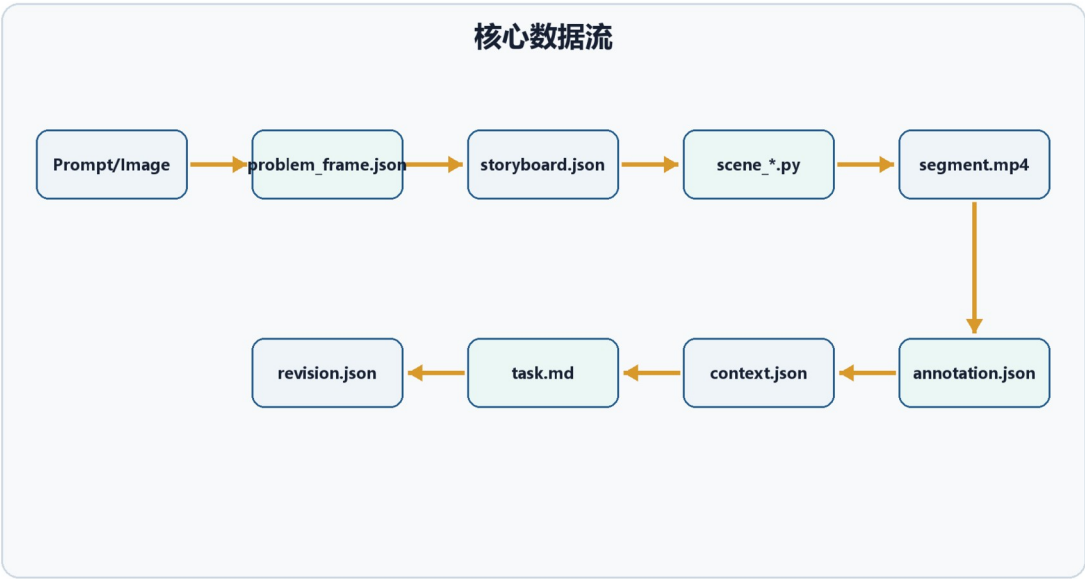


图 2-2 从课程主题到修订记录的核心数据流

第三章 系统总体设计

3.1 总体架构

平台采用本地客户端/服务端混合架构。项目 1 由 Electron 承担桌面交互与进程管理，FastAPI 暴露本地服务，Python 服务层负责任务编排，ManimCE 执行渲染；项目 2 由 React 审阅页面和独立 FastAPI 服务组成，读取项目视频与 shot_map 并生成结构化审阅资料。两个项目不共享运行进程，但共享相同的资产思想：分镜是最小教学单元，文件是跨模块契约，日志与版本是质量保障基础。

这种架构的优点是渲染和前端解耦，长任务不会阻塞界面；生成端可以独立批量生产，审阅端可以对已有项目反复检查。代价是本地服务端口、Python 环境和前端资源需要统一启动管理，且项目 1 与项目 2 之间目前仍以目录导入和文件复制为主，尚未形成单一项目协议。



图 3-1 课程设计端到端工作流程

3.2 项目 1 分层设计

表 3-1 项目 1 分层架构

层次	主要目录/技术	职责
表现层	electron/	快速生成、节点画布、风格库、批注与预览
接口层	backend/main.py · FastAPI	参数接收、任务提交、状态查询、文件服务
编排层	services/generation_service.py	大纲、分镜、代码、渲染、音频的阶段调度
工作流层	workflow/	节点定义、DAG 校验、拓扑执行与输出快照
模型层	ai/	配置能力、提示词、结构化输出与本地离线降级
渲染层	rendering/	代码清洗、静态检查、视觉守卫、Manim 进程
资产层	generated_projects/ · config/	项目成果、风格、日志、修订和媒体

3.3 项目 2 分层设计

项目 2 被定位为“Agent-in-the-loop 审阅伴侣”而非独立 AI 应用。前端负责视频播放、时间轴、画布批注、任务展示和修订操作；后端负责批注持久化、时间匹配、代码区域提取、上下文打包和版本操作；Skill 定义 Agent 必须遵守的修改、渲染和回滚规则。智能修改发生在 Agent 侧，因此审阅后端不保存模型提供商，也不存在 llm_client.py。

3.4 项目目录结构



图 3-2 项目 1 与项目 2 的目录结构

如图 3-2 所示，两个项目都把运行代码、配置、项目资产和脚本分开。项目 2 的 projects/demo_project 进一步按照 source、shots、context、agent_tasks、revisions 和 versions 组织，使一次批注从输入到修改结果都有确定落点。

第四章 系统功能设计

4.1 快速生成模式

快速生成模式面向不希望手工搭建节点的用户。界面集中提供桥梁主题提示词、参考图片、模型能力配置、渲染质量、总时长、分镜数量、紧凑节奏和输出目录。提交后后端创建异步任务，前端轮询任务注册表并显示当前阶段、进度、剩余阶段、音频状态和日志。生成完成后可

查看分镜、代码、修复记录与项目文件。



图 4-1 项目 1 快速生成与片段管理界面

4.2 节点工作流模式

节点模式把输入、参考、规划、控制、逻辑、代码、渲染和输出拆成可连接单元。每个端口具有明确数据类型，例如 Prompt、StoryboardJSON、ManimCode 和 VideoFile；Any 端口可接收任意类型。验证器检查未知节点、必填输入、端口方向、类型兼容与环路，执行器按照拓扑序运行并把节点状态、输入快照和输出写入项目目录。

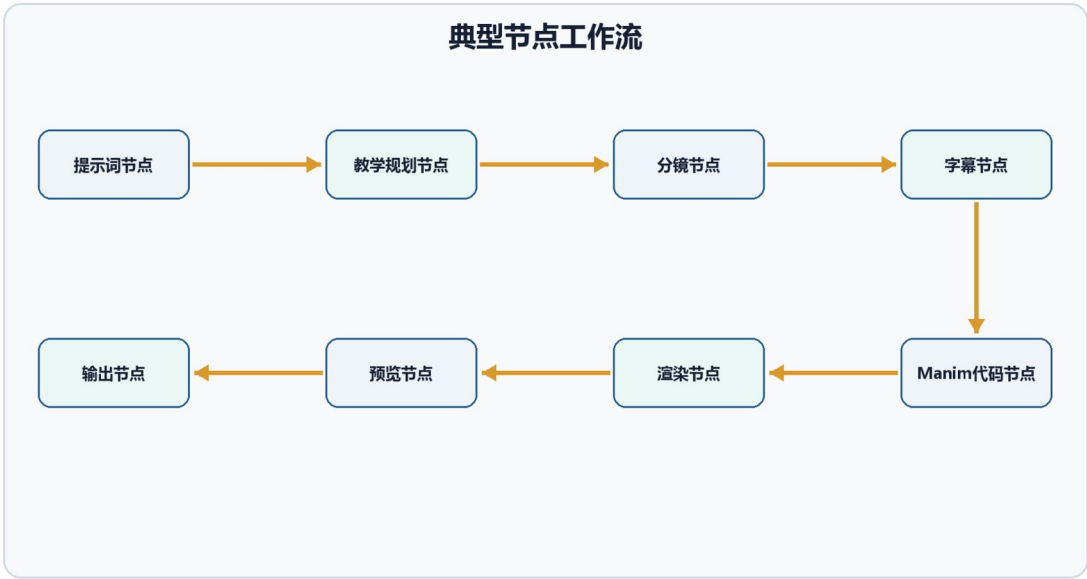


图 4-2 典型节点 workflow 及数据依赖

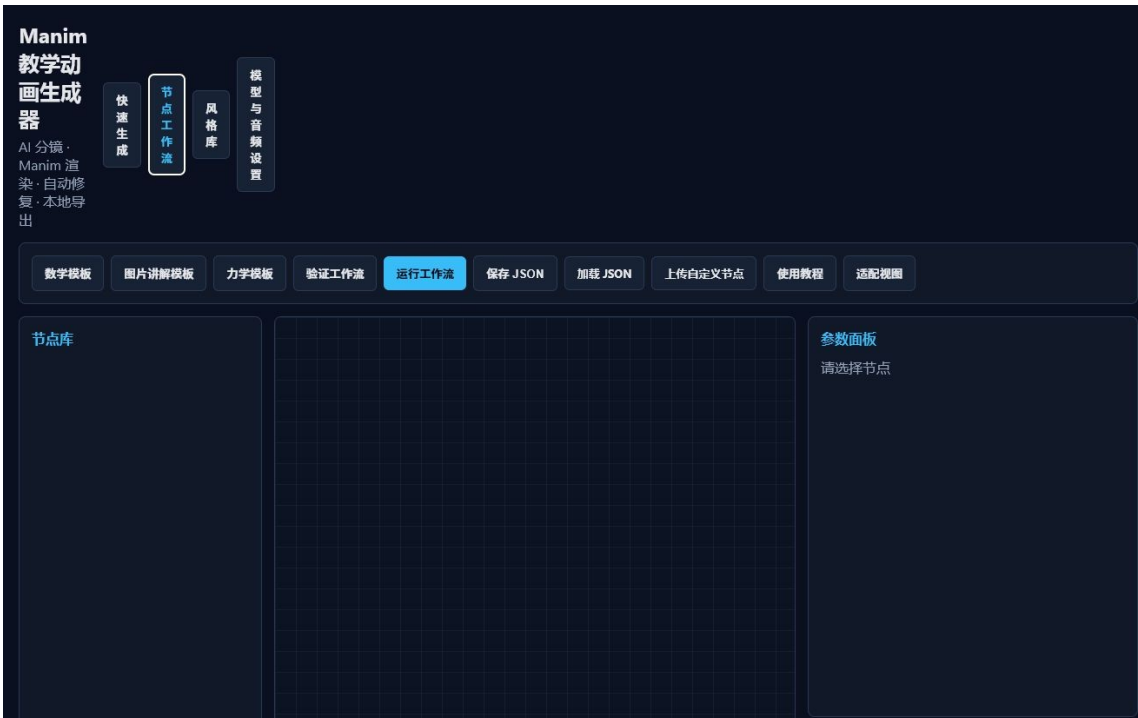


图 4-3 项目 1 节点 workflow 编辑界面

4.3 分镜生成与管理

分镜不是单纯的字幕切片，而是桥梁知识的最小可执行教学单元。每个分镜包含教学目的、视觉计划、旁白、场景类和资产路径。系统允许在生成后选择片段，读取当前 Manim 代码，上传或编辑.py 文件，并仅重渲染当前片段。历史代码版本可载入，音画时长策略可选自动适配、保持原始时长或手动时长。

4.4 模型、风格与音频配置

项目 1 实现模型能力画像和配置档案，字段覆盖文本、视觉、图片上传、多模态、函数调用、JSON 输出和流式能力；风格库保存颜色、构图、节奏和动画语言，并支持分析、导入、更新及版本回滚。音频服务支持片段级缓存、项目级拼接、音视频混流和失败日志。课程设计便携离线版已将模型路由强制转为本地规则生成并关闭远程 TTS，证明配置层与业务层可以解耦。

4.5 批注与交互审阅

项目 2 允许用户在暂停的视频帧上绘制矩形、箭头或自由曲线，为每个图形附加稳定编号和局部标签，同时填写全局修改要求。提交截图不是透明画布，而是“视频帧+批注层”的合成图。后端依据 `video_time` 在 `shot_map` 中匹配分镜，把批注、对象清单、源码行区间、渲染命令和回滚命令共同写入上下文包。

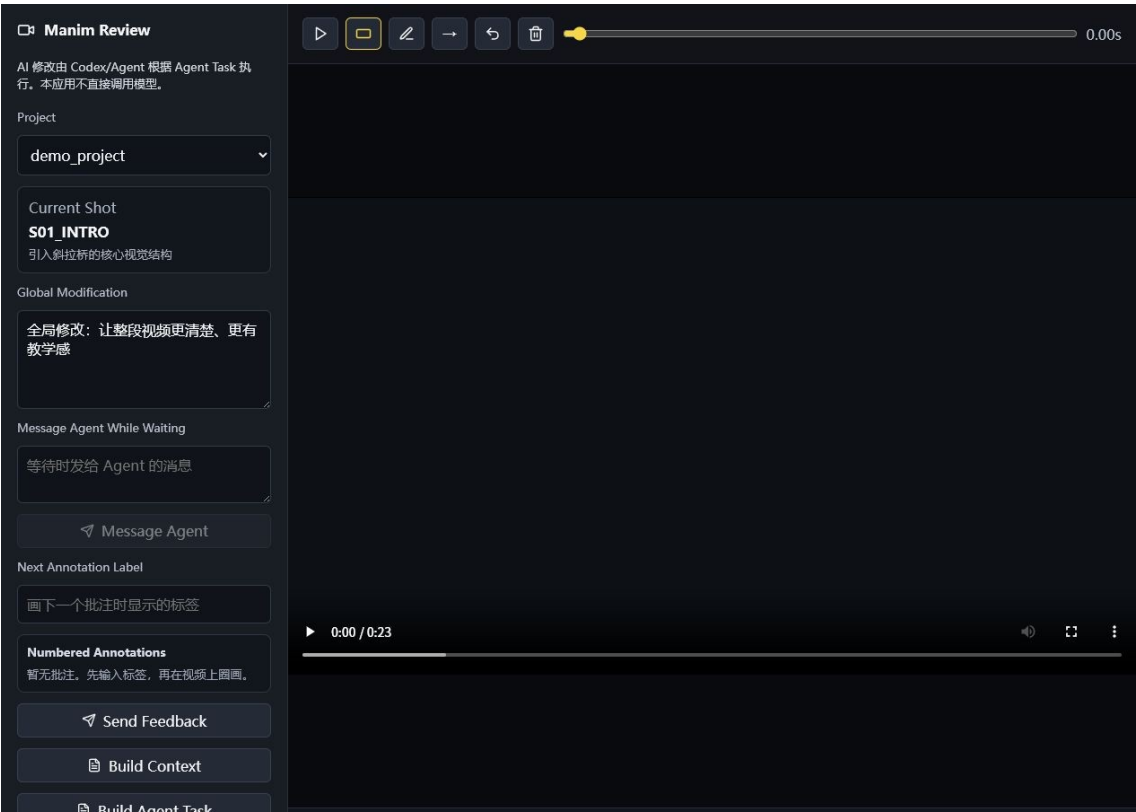


图 4-4 项目 2 交互式视频审阅界面

4.6 修订接受与回滚

Agent 完成局部修改后，`finalize_agent_patch.py` 依次执行目标分镜渲染、完整预览更新和修订记录创建，只有前两步成功才产生正式修订。用户可以接受修订，也可以调用 `rollback` 恢复前一版本的 `source` 和 `shot_map`。浏览器通过项目版本与 `review-status` 轮询自动刷新视频，并在刷新

前收集尚未提交的批注，避免反馈丢失。

第五章 核心模块实现

5.1 问题定框与流水线记录

生成服务首先把输入整理为 `problem_frame.json`，记录提示词摘要、图片状态、冲突优先级、目标时长、质量与紧凑节奏。该文件在后续大纲、分镜和代码阶段保持不变，防止多轮生成逐渐偏离桥梁主题。Pipeline Recorder 维护 `pipeline_manifest.json` 和追加式 `pipeline_events.jsonl`，分别描述阶段状态与事件序列。

表 5-1 生成流水线阶段设计

阶段	输入	主要输出	失败处理
prepare	主题、素材、参数	<code>problem_frame.json</code>	输入校验并终止
outline	问题定框	教学大纲	结构化解析或降级
storyboard	大纲、视觉风格	分镜 JSON	批量重试与字段校验
codegen	分镜、视觉计划	scene Python	代码清洗
static_check	Python 源码	检查日志	进入修复闭环
render_course	合格源码	片段视频	超时或错误归因
stitch/export	片段、字幕、音频	最终视频与清单	保留可用中间件

5.2 结构化分镜与视觉计划

大纲阶段负责教学顺序，分镜阶段负责把顺序变为可执行画面。针对斜拉桥示例，第一镜建立桥面—桥塔—扇形斜拉索的总体认知；第二镜使用向下荷载箭头和高亮构件解释传力路径；第三镜归纳“斜拉索受拉、桥塔受压、无跨全桥主缆”等要点。该设计将桥梁工程语义落实为对象、颜色、运动和时长，而不是把一段文字直接塞入画面。

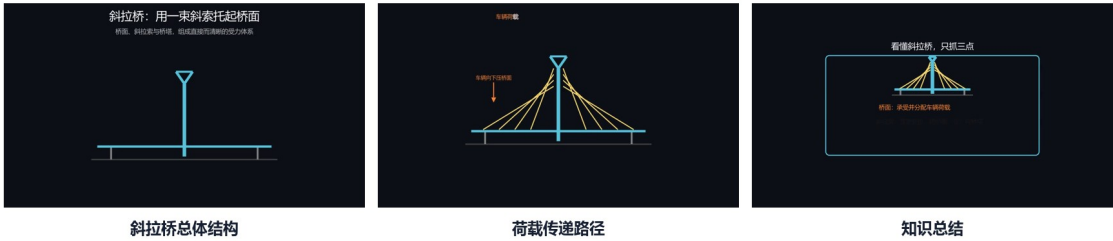


图 5-1 斜拉桥三分镜运行效果

5.3 Manim 代码生成与清洗

代码生成器要求输出 Manim Community Edition 语法，并以独立 Scene 承载分镜。Code Sanitizer 移除 Markdown 围栏、解释性前后缀和危险片段，规范入口场景名称。Static Checker 先调用 `py_compile`，必要时再执行可选名称检查；Visual Guard 通过主题词、占位图、坐标轴或矩阵模板等规则检测视觉方案是否误用旧主题资产。

5.4 渲染与自动修复

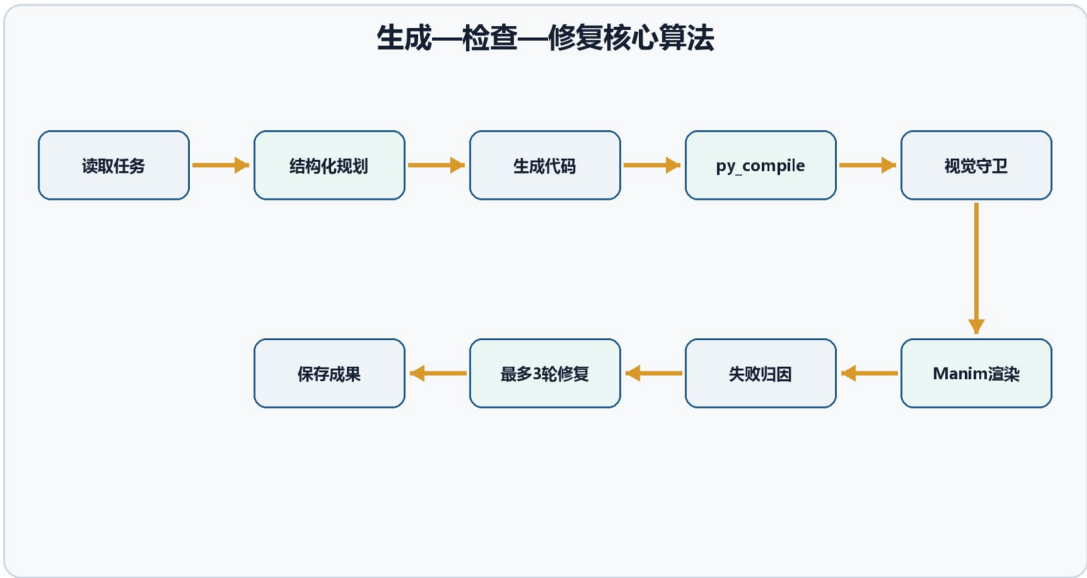


图 5-2 生成、检查、渲染和修复算法

如图 5-2 所示，静态错误不启动 Manim，直接携带错误、当前代码、教学目标和 `visual_plan` 进入修复；渲染错误则截取进程输出并进入同一闭环。最大修复轮数默认为 3，避免无穷重试。成功后记录命令、耗时、视频路径和修复历史；失败时保留源码及日志，便于人工定位。

5.5 节点执行器

工作流程验证通过后，执行器根据边集计算入度并进行拓扑排序。对每个节点，先汇聚上

游输出与本节点参数，再调用对应处理器；执行状态依次经历未开始、等待输入、排队、运行、完成、失败或跳过。条件节点把数据送入 true/false 端口，多分支节点复制输入，合并节点按照策略聚合。任何参数变化都会使该节点及下游状态失效，保证重新执行使用新数据。

5.6 批注定位与上下文构建

项目 2 的 `match_shot` 遍历 `shot_map`，当 `start_time ≤ video_time ≤ end_time` 时返回目标 `shot`。`build_context` 读取该 `shot` 的 `scene_class` 和源文件，通过类定义边界提取代码区域及起止行；随后组合用户全局要求、编号批注标签、截图路径、对象清单、渲染命令和规则。上下文包既给 Agent 足够信息，又避免加载整个视频源码。



图 5-3 交互批注到局部修订的闭环

5.7 版本控制实现

修订前，`backup_version` 把 `source` 与 `shot_map` 复制到 `versions/vNNN`；`create_revision` 写入 `from_version`、`to_version`、`shot_id`、`changed_files`、`render_status`、`summary` 和 `context_package`；`rollback` 根据当前版本计算上一版本，恢复源码和分镜表并写回 `current.json`。示例项目当前已迭代至 `v014`，说明版本机制能够承载连续审阅。

第六章 关键技术分析

6.1 工程知识向视觉语言的转换

桥梁动画的难点不在于画出桥，而在于建立工程意义与视觉动作之间的映射。例如“斜拉索受拉”应通过索色高亮、沿索方向的拉力箭头和构件标签共同表达；“桥塔受压”应采用竖

向力流与塔柱强调；“荷载传至地基”需要画面顺序连续，不能让箭头瞬间跳跃。系统通过教学大纲、visual_plan 和对象清单三层描述降低语义损失。

6.2 DAG 验证与类型系统

节点 workflow 本质上是有向无环图 $G=(V,E)$ 。拓扑执行要求图中不存在从节点回到自身的路径；端口连接还需满足类型相同或一端为 Any。验证发生在运行前，可把许多运行时错误转化为清晰的编辑提示。其优点是组合灵活、依赖可视化；不足是节点粒度过细会增加画布复杂度，因此系统同时保留快速生成模式。

6.3 异步任务和状态恢复

视频生成耗时远大于普通 HTTP 请求。项目 1 通过 TaskRegistry 返回 task_id，后台协程更新阶段、进度、消息和结果，前端轮询状态。暂停与继续由任务控制点协同实现。任务状态写入项目目录后，后端重启可识别已中断任务并标记异常，而不是继续显示“运行中”。

6.4 音画同步

片段媒体服务分别记录原始视频、修正视频、音频源和预览路径。自动适配策略以旁白音频时长为目标调整片段节奏，保持策略保留 Manim 原始时长，手动策略接受用户秒数。音频按分镜缓存，项目级合成时先拼接再通过 FFmpeg 混流；单个片段失败不会删除静音视频。离线版本关闭远程合成，但保留已有音频与本地媒体处理能力。

6.5 局部修改边界

局部修改采用三重边界：时间边界由 shot_map 确定，代码边界由 scene_class 起止行确定，对象边界由 OBJECT_ID 提示。Agent 任务明确禁止重写整部视频、删除标识或引入 ManimGL 语法。与直接把整份源代码交给模型相比，该方法减少上下文量，降低无关画面被改坏的概率，也让渲染失败后的回滚更明确。

6.6 本地化与可审计性

项目采用 localhost 通信、文件化数据契约和便携运行时。项目 2 明确不在前后端直接调用大模型，项目 1 的离线发行版将外部模型配置强制路由为本地生成并关闭远程 TTS。所有关键输入、输出和错误均落盘，使课程设计成果可在无网络条件下检查，也避免 API 密钥进入工作流 JSON 或提交包。

第七章 系统运行与测试

7.1 运行环境与启动方式

表 7-1 系统运行环境

项目	前端/桌面	后端	关键运行时
项目 1	Electron 31	FastAPI + Uvicorn	Python 3.12、ManimCE 0.20.1、FFmpeg
项目 2	React 18 + Vite 5	FastAPI + 文件工作流	ManimCE 0.20.1、浏览器 Canvas

项目 1 便携版将 Electron、Python 和 Manim 运行时放在同一目录，由 Start-Manium.cmd 设置 PYTHONHOME、PYTHONPATH 和本地离线模式后启动。项目 2 可通过启动审阅脚本检查依赖、启动 8000 端口后端与 5173 端口前端，并打开 demo_project。

7.2 测试方法



图 7-1 系统分层测试策略

表 7-2 主要测试结果

测试类别	代表性内容	结果
项目 1 自动化测试	批注、工作流、配置能力、风格	46/46 通过

	库、任务监控、片段编辑等 46 项	
项目 1 编译检查	backend 全模块 compileall	通过
项目 1 便携校验	Electron、Python、Manim、启动器、密钥排除	通过
项目 2 静态自检	目录、脚本、Skill、预览视频	通过
项目 2 接口测试	video=200、批注保存、shot 匹配、上下文和任务生成	通过
项目 2 渲染测试	S02 分镜低质量渲染与 finalize 流程	通过
项目 2 版本测试	修订列表、接受与 rollback 恢复	通过
项目 2 前端测试	Vite 构建与浏览器加载	通过

7.3 斜拉桥示例验证

示例视频总长约 23 s，shot_map 包含 S01_INTRO（0.0 ~ 5.8 s）、S02_FORMULA_EXPLAIN（5.8~17.0 s）和 S03_SUMMARY（17.0~23.0 s）。在第二分镜任意时刻提交批注，后端应匹配 S02 并把 formula_main、bridge_structure 和 explanation_arrow 写入对象清单。该用例同时验证时间边界、桥梁语义对象、代码局部提取与渲染命令生成。

7.4 测试结论

测试覆盖了“能否运行”“能否生成”“失败能否定位”“修改能否回退”四个层次。自动化结果证明基础服务稳定；实际页面和视频帧证明成果可见；版本与回滚验证证明系统不会以不可逆方式修改课程成果。尚未覆盖的部分主要是大规模长视频压力、跨机器字体一致性和自动视觉质量评分，将在第九章提出改进。

第八章 项目特色与创新点

8.1 双系统闭环而非单次生成

本项目没有把“生成视频”视为终点，而是把生成端与审阅端组合成闭环。项目 1 提高首次成片效率，项目 2 把教师或学生的视觉反馈转换为可执行局部任务，二者分别优化批量生产

和精细修改。

8.2 面向桥梁工程的分层视觉设计

通过“教学目标—分镜头—一视觉计划—对象标识—动画代码”逐层细化，桥梁工程概念能够落到具体构件与动作。斜拉桥示例不是通用几何动画套壳，而是围绕桥面、桥塔、斜拉索和地基的受力关系设计。

8.3 可视化节点与快速模式并存

节点 workflow 适合展示课程设计的算法和数据依赖，快速模式适合直接生产。两种入口复用同一生成与渲染基础能力，兼顾教学可解释性和实际操作效率。

8.4 编号批注与代码对象映射

批注系统区分全局修改要求和局部编号标签，截图保留真实视频帧，任务保留 `OBJECT_ID` 和代码行区间。该设计把“画面上这里需要改”转化为机器可理解的局部约束，是项目最具辨识度的交互创新。

8.5 失败可见与可回退

静态检查、渲染日志、最多三轮修复、修订记录和版本回滚共同构成防护网。系统不隐藏失败，也不在渲染失败时创建成功修订，符合工程软件对可追溯性和结果可信度的要求。

第九章 存在的问题与改进方向

9.1 工程计算能力边界

当前系统擅长结构概念与传力路径表达，但没有实现荷载组合、内力计算、截面验算或有限元求解。因此报告中的动画只能作为设计说明和教学辅助，不能作为桥梁安全判定依据。后续可引入规范参数表、结构计算模块或与有限元软件导出结果对接，并在画面中明确数据来源与单位。

9.2 自动视觉质量评价不足

Visual Guard 主要在渲染前检查代码和主题一致性，尚不能可靠发现文字遮挡、对象越界、对比度不足、空白帧和字幕过快。可在渲染后按时间抽帧，结合几何边界、OCR 和颜色对比算法形成质量评分；对长视频应按分镜并行抽帧，避免整片分析成本过高。

9.3 两项目资产协议尚未完全统一

项目 2 的 `shot_map`、`SHOT_ID` 和 `OBJECT_ID` 体系与项目 1 的分镜资产概念相近，但当前仍需要导入或整理。建议定义统一 `project_manifest.json`，固定视频、分镜、源码、对象、音频和版本字段，使项目 1 生成结果可一键进入项目 2 审阅。

9.4 便携性和依赖体积

完整 Electron、Python、Manim、LaTeX 与媒体工具会使便携包较大。可通过剔除开发缓存按需打包字体和 TeX 组件、把示例工程与运行时分离来减小体积；同时应保留哈希校验和环境自检，避免过度裁剪导致换机后无法渲染。

9.5 交互与协同扩展

当前审阅以单机文件为中心，适合课程设计个人迭代。后续可增加多人批注、批注状态、教师审核意见、差异对比和只读发布模式。对于 Agent 修改，可增加自动生成前后关键帧对比和修改影响范围报告，降低人工复核负担。

表 9-1 后续改进路线

优先级	改进项	预期收益
高	统一两个项目的 <code>project_manifest</code> 与 <code>shot schema</code>	实现生成后直接审阅
高	渲染后抽帧质量检测	自动发现遮挡、越界和空白
中	引入桥梁计算结果接口	增强工程数据真实性
中	修订前后可视化差异	提高教师验收效率
低	多人协同和权限	支持课程小组与教师评阅

第十章 总结与展望

10.1 课程设计总结

本课程设计完成了一个由项目 1 和项目 2 共同组成的桥梁工程教学动画平台。项目 1 建立了从问题定框到视频导出的生产流水线，并以节点 workflow、静态检查、视觉守卫和修复闭环提高可解释性与稳定性；项目 2 建立了从视频批注到局部修订的审阅流水线，并以分镜匹配、对象

标识、上下文包、Agent Task 和版本回滚控制修改范围。

在实现过程中，设计重点从“调用模型生成代码”逐步转向“建立约束、记录状态、暴露失败、允许回退”。这一转变使系统更符合工程课程设计的严谨性：最终视频只是成果之一，分镜、代码、日志、批注和修订同样是设计过程的组成部分。斜拉桥示例验证了平台对桥梁构造关系和荷载传递的动态表达能力。

10.2 展望

未来工作将围绕工程数据接入、统一项目协议、视觉质量检测和协同审阅展开。若能进一步对接桥梁计算结果与规范条文，系统可从“工程知识动画工具”发展为“设计计算—视觉解释—交互复核”一体化平台。与此同时，仍应坚持人工校核：AI 和动画负责组织与表达，结构安全结论必须由规范、计算和专业判断共同保证。

参考文献

- [1] 中华人民共和国交通运输部. 公路桥涵设计通用规范: JTG D60—2015[S]. 北京: 人民交通出版社, 2015.
- [2] 邵旭东, 程翔云, 李立峰. 桥梁工程[M]. 北京: 人民交通出版社.
- [3] Manim Community Developers. Manim Community Edition Documentation[EB/OL]. <https://docs.manim.community/>.
- [4] FastAPI. FastAPI Documentation[EB/OL]. <https://fastapi.tiangolo.com/>.
- [5] Electron. Electron Documentation[EB/OL]. <https://www.electronjs.org/docs/latest/>.
- [6] React Team. React Documentation[EB/OL]. <https://react.dev/>.
- [7] Python Software Foundation. Python 3 Documentation[EB/OL]. <https://docs.python.org/3/>.
- [8] FFmpeg Developers. FFmpeg Documentation[EB/OL]. <https://ffmpeg.org/documentation.html>.
- [9] 项目源码. Manim 教学动画生成器: ARCHITECTURE.md、WORKFLOW_TUTORIAL.md 及 backend 模块[Z]. 2026.
- [10] 项目源码. Manim Interactive Review: README.md、SELF_CHECK_REPORT.md 及 manim_interactive_review Skill[Z]. 2026.

致 谢

感谢指导教师在桥梁工程课程设计目标、工程表达规范和成果组织方面给予的指导；感谢课程学习中使用的桥梁工程教材、开源 Manim 社区以及 Python、FastAPI、Electron 和 React 生态提供的技术基础。项目开发过程中形成的失败日志、测试记录和多轮修改也为本报告的总结提供了重要依据。